

SzalkaCinema – Fejlesztői dokumentáció

Verzió: 1.0

Utolsó frissítés: 2026. február 25.

Tartalomjegyzék

1. Bevezetés és projekt áttekintés
 2. Rendszerarchitektúra
 3. Telepítési útmutató
 4. Adatbázis struktúra
 5. Modulok és funkciók részletes leírása
 6. API és külső szolgáltatások
 7. Biztonsági megfontolások
 8. Hibakeresés és naplózás
 9. Teljesítmény optimalizálás
 10. Kódolási konvenciók
-

1. Bevezetés és projekt áttekintés

1.1 A projekt célja

A SzalkaCinema egy teljes körű mozijegy-foglalási rendszer, amely lehetővé teszi a felhasználók számára, hogy:

- Böngésszenek a moziban vetített filmek között
- Megtekintsék a filmek részletes adatait (leírás, rendező, hossz, műfaj, értékelés)
- Elolvassák és értékeljék a filmeket (kommentek, csillagos értékelés)
- Megtekintsék a heti műsort napokra bontva
- Szűrjenek vetítéseket dátum és filmcím alapján
- Kiválasszák a kívánt helyeket egy interaktív ülésrendben
- Fizessenek online bankkártyával (Stripe integráció)
- Kezeljék a megvásárolt jegyeiket
- Adminisztrátorként teljes körűen kezeljék a filmeket, vetítéseket, felhasználókat, jegyeket és kommenteket

1.2 Technológiai stack

Összetevő	Technológia
Backend	PHP 7.4+ (natív, keretrendszer nélkül)
Adatbázis	MySQL 5.7+ / MariaDB 10.2+
Frontend	HTML5, CSS3, JavaScript (natív)
Fizetési rendszer	Stripe API v19.3
Szerver	Apache / Nginx
Verziókezelés	Git

1.3 Rendszerkövetelmények

Szerver oldali követelmények:

- PHP 7.4 vagy újabb
- MySQL 5.7 vagy újabb / MariaDB 10.2 vagy újabb
- Apache vagy Nginx webservert
- SSL tanúsítvány (éles környezetben kötelező)
- Composer (függőségkezeléshez)
- cURL bővítmény
- PDO MySQL bővítmény
- JSON bővítmény

Kliens oldali követelmények:

- Modern webböngésző (Chrome, Firefox, Safari, Edge legfrissebb verziói)
- JavaScript engedélyezve
- Cookie-k engedélyezve
- Internetkapcsolat

1.4 Mappa struktúra

szalkacinema/

— admin.php	# Admin felület
— admin_action.php	# Admin műveletek
— booking.php	# Jegyfoglalás
— composer.json	# Composer függőségek
— composer.lock	# Composer lock fájl
— config.php	# Fő konfigurációs fájl
— footer.php	# Lábléc sablon

— header.php	# Fejléc sablon
— index.php	# Főoldal / Heti műsor
— login.php	# Bejelentkezés
— logout.php	# Kijelentkezés
— movie.php	# Film részletei
— movies.php	# Filmek listája
— register.php	# Regisztráció
— screenings.php	# Vetítések listája
— stripe_payment.php	# Stripe fizetés indítása
— stripe_success.php	# Stripe siker oldal
— style.css	# Globális stíluslap
— test_stripe.php	# Stripe tesztelő
— tickets.php	# Jegyek kezelése
— vendor/	# Composer által telepített csomagok
— README.md	# Projekt leírás

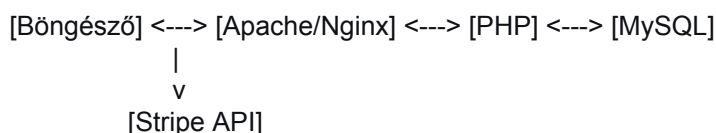
2. Rendszerarchitektúra

2.1 Általános architektúra

A SzalkaCinema egy hagyományos, szerveroldali webalkalmazás, amely MVC-szerű mintát követ, de nem használ keretrendszert. Az architektúra három fő rétegből áll:

1. **Prezentációs réteg** – HTML, CSS, JavaScript (header.php, footer.php, style.css)
2. **Üzleti logika réteg** – PHP fájlok (index.php, movies.php, booking.php stb.)
3. **Adatbázis réteg** – MySQL (PDO kapcsolaton keresztül)

2.2 Kommunikációs folyamat



2.3 Munkamenet kezelés

A rendszer PHP session-öket használ a felhasználói munkamenetek kezelésére. A session a bejelentkezés után tárolja a felhasználó azonosítóját, amely alapján minden oldalbetöltéskor lekérdezésre kerül a felhasználó aktuális adatai.

```

// config.php
session_start();

function getCurrentUser() {
    if (isset($_SESSION['user_id'])) {
        global $pdo;
        $stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
        $stmt->execute([$_SESSION['user_id']]);
    }
}

```

```
        return $stmt->fetch(PDO::FETCH_ASSOC);
    }
    return null;
}
```

2.4 Automatikus vetítés generálás

A rendszer egyedi funkciója, hogy automatikusan generálja a napi és heti vetítéseket, ha azok hiányoznak:

- **generateDailyScreenings()** – Ellenőrzi, hogy vannak-e vetítések az adott napra, és ha nincs legalább 5, akkor generál véletlenszerűen 5 vetítést a napra.
- **generateWeeklyScreenings()** – Ugyanezt végzi el 7 napra előre.

Ez a funkció biztosítja, hogy mindig legyenek elérhető vetítések, még akkor is, ha az adminisztrátor nem töltötte fel manuálisan.

3. Telepítési útmutató

3.1 Előfeltételek ellenőrzése

```
php -v
mysql --version
composer --version
```

3.2 Telepítés lépései

3.2.1 Forráskód letöltése

```
git clone https://github.com/your-repo/szalkacinema.git
cd szalkacinema
```

3.2.2 Függőségek telepítése

```
composer install
```

3.2.3 Adatbázis létrehozása

```
CREATE DATABASE mozivizsga26 CHARACTER SET utf8mb4 COLLATE utf8mb4_hungarian_ci;
CREATE USER 'mozivizsga26'@'localhost' IDENTIFIED BY 'Mozivizsga2026!';
GRANT ALL PRIVILEGES ON mozivizsga26.* TO 'mozivizsga26'@'localhost';
FLUSH PRIVILEGES;
```

3.2.4 Adatbázis táblák létrehozása

Felhasználók tábla:

```
CREATE TABLE users (
```

```

id INT AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(50) UNIQUE NOT NULL,
email VARCHAR(100) UNIQUE NOT NULL,
password VARCHAR(255) NOT NULL,
full_name VARCHAR(100) NOT NULL,
role ENUM('user', 'admin') DEFAULT 'user',
registration_date DATETIME DEFAULT CURRENT_TIMESTAMP,
last_login DATETIME NULL,
INDEX (username),
INDEX (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Filmek tábla:

```

CREATE TABLE movies (
id INT AUTO_INCREMENT PRIMARY KEY,
title VARCHAR(255) NOT NULL,
description TEXT NOT NULL,
director VARCHAR(100),
duration INT NOT NULL,
release_year INT,
genre VARCHAR(100),
rating DECIMAL(3,1),
poster_url VARCHAR(500),
trailer_url VARCHAR(500),
active BOOLEAN DEFAULT TRUE,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
INDEX (title),
INDEX (active)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Vetítések tábla:

```

CREATE TABLE screenings (
id INT AUTO_INCREMENT PRIMARY KEY,
movie_id INT NOT NULL,
screening_date DATE NOT NULL,
screening_time TIME NOT NULL,
hall_number INT NOT NULL,
price DECIMAL(10,2) NOT NULL,
available_seats INT NOT NULL,
FOREIGN KEY (movie_id) REFERENCES movies(id) ON DELETE CASCADE,
INDEX (screening_date),
INDEX (movie_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Jegyek tábla:

```

CREATE TABLE tickets (
id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
screening_id INT NOT NULL,
seat_number VARCHAR(10) NOT NULL,
price_paid DECIMAL(10,2) NOT NULL,
payment_id INT NULL,
payment_status VARCHAR(20) DEFAULT 'pending',
status VARCHAR(20) DEFAULT 'active',
purchase_date DATETIME DEFAULT CURRENT_TIMESTAMP,

```

```

FOREIGN KEY (user_id) REFERENCES users(id),
FOREIGN KEY (screening_id) REFERENCES screenings(id),
UNIQUE KEY unique_seat (screening_id, seat_number),
INDEX (user_id),
INDEX (screening_id),
INDEX (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Kommentek tábla:

```

CREATE TABLE comments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  movie_id INT NOT NULL,
  comment TEXT NOT NULL,
  rating INT NOT NULL CHECK (rating >= 1 AND rating <= 5),
  comment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (movie_id) REFERENCES movies(id) ON DELETE CASCADE,
  INDEX (movie_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Fizetések tábla:

```

CREATE TABLE payments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  screening_id INT NOT NULL,
  stripe_session_id VARCHAR(255) NOT NULL,
  stripe_payment_intent VARCHAR(255) NULL,
  amount INT NOT NULL,
  seats JSON NOT NULL,
  status VARCHAR(20) DEFAULT 'pending',
  paid_at DATETIME NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (screening_id) REFERENCES screenings(id),
  INDEX (user_id),
  INDEX (screening_id),
  INDEX (stripe_session_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

3.2.5 Konfiguráció beállítása

Szerkessze a config.php fájlt az adatbázis és Stripe adatokkal:

```

// Adatbázis kapcsolat
$host = 'localhost';
$dbname = 'mozivizsga26';
$username = 'mozivizsga26';
$password = 'Mozivizsga2026!';

// Stripe API kulcsok
define('STRIPE_PUBLISHABLE_KEY', 'pk_test_...');
define('STRIPE_SECRET_KEY', 'sk_test_...');

```

3.2.6 Tesztadatok feltöltése (opcionális)

-- Admin felhasználó létrehozása (jelszó: admin123)

```
INSERT INTO users (username, email, password, full_name, role) VALUES  
( 'admin', 'admin@szalkacinema.hu', '$2y$10$YourHashedPasswordHere', 'Adminisztrátor', 'admin');
```

-- Teszt filmek

```
INSERT INTO movies (title, description, director, duration, release_year, genre, rating, poster_url)  
VALUES  
( 'A keresztapa', 'A Vito Corleone vezette maffiacsalád története...', 'Francis Ford Coppola', 175, 1972,  
'Dráma, Krimi', 9.2, 'https://example.com/godfather.jpg'),  
( 'A sötét lovag', 'Batmannek szembe kell néznie a Jokerrel...', 'Christopher Nolan', 152, 2008, 'Akció,  
Krimi', 9.0, 'https://example.com/darkknight.jpg');
```

3.2.7 Webszerver konfiguráció

Apache (.htaccess):

```
RewriteEngine On  
RewriteCond %{REQUEST_FILENAME} !-f  
RewriteCond %{REQUEST_FILENAME} !-d  
RewriteRule ^(.*)$ index.php [QSA,L]  
  
# Biztonsági fejlécek  
Header set X-Content-Type-Options "nosniff"  
Header set X-Frame-Options "DENY"  
Header set X-XSS-Protection "1; mode=block"
```

Nginx (virtual host):

```
server {  
    listen 80;  
    server_name szalkacinema.local;  
    root /var/www/szalkacinema;  
  
    index index.php;  
  
    location / {  
        try_files $uri $uri/ /index.php?$args;  
    }  
  
    location ~ \.php$ {  
        include snippets/fastcgi-php.conf;  
        fastcgi_pass unix:/var/run/php/php7.4-fpm.sock;  
    }  
  
    # Biztonsági fejlécek  
    add_header X-Content-Type-Options "nosniff" always;  
    add_header X-Frame-Options "DENY" always;  
    add_header X-XSS-Protection "1; mode=block" always;  
}
```

3.3 Telepítés ellenőrzése

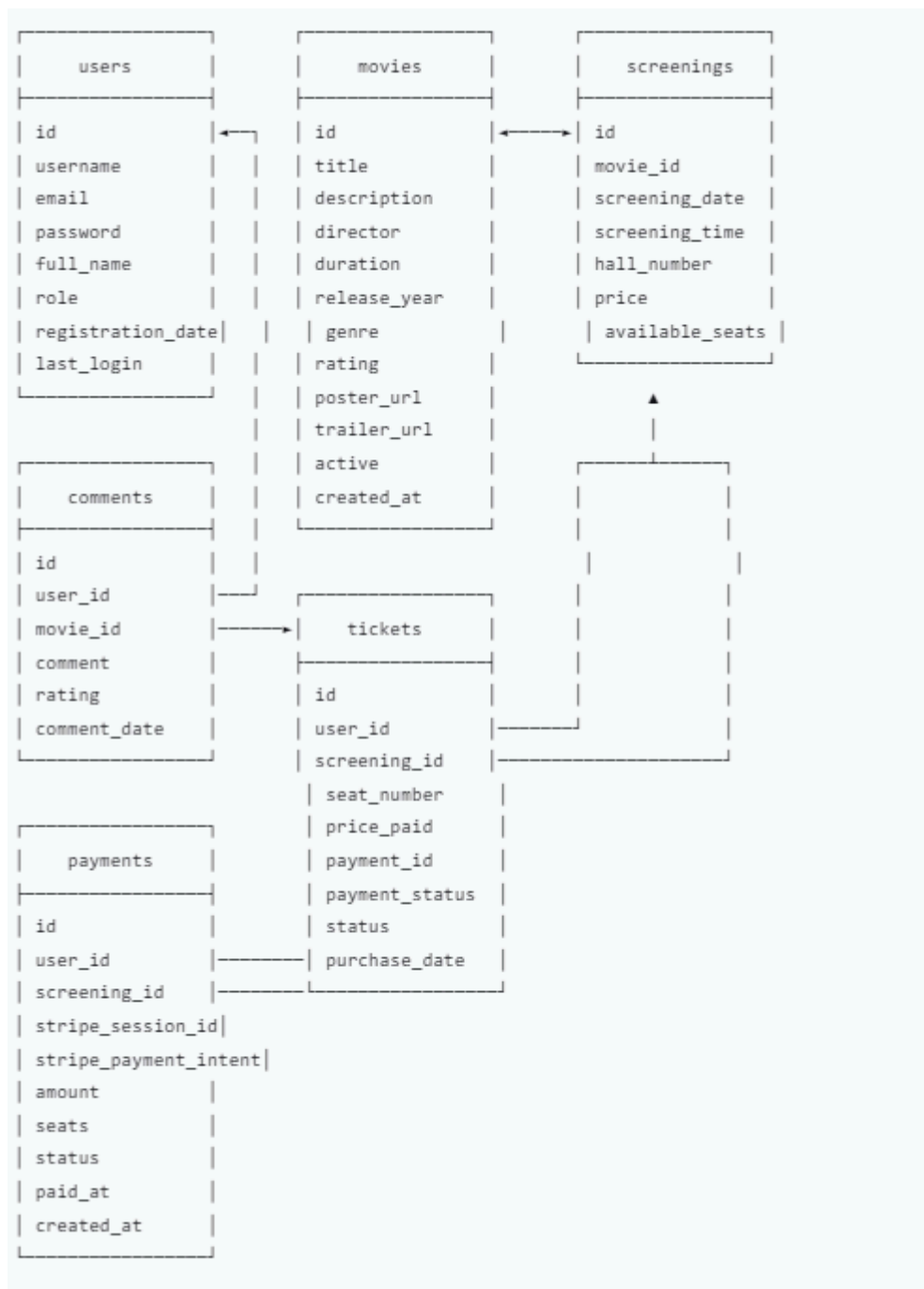
Nyissa meg a böngészőben: <http://localhost/mozi/>

Az oldalnak meg kell jelennie. Ellenőrizze az alábbiakat:

- Főoldal betöltődik
- Heti műsor megjelenik
- Regisztráció működik
- Bejelentkezés működik
- Filmek listája betöltődik
- Vetítések listája betöltődik

4. Adatbázis struktúra

4.1 Adatbázis diagram



4.2 Táblák részletes leírása

4.2.1 users – Felhasználók

Mező	Típus	Leírás	Alapértelmezett
id	INT AUTO_INCREMENT	Egyedi azonosító	-
username	VARCHAR(50)	Felhasználónév (egyedi)	-
email	VARCHAR(100)	Email cím (egyedi)	-
password	VARCHAR(255)	Hashelt jelszó	-
full_name	VARCHAR(100)	Teljes név	-
role	ENUM('user','admin')	Szerepkör	'user'
registration_date	DATETIME	Regisztráció időpontja	CURRENT_TIMESTAMP
last_login	DATETIME	Utolsó bejelentkezés	NULL

4.2.2 movies – Filmek

Mező	Típus	Leírás	Alapértelmezett
id	INT AUTO_INCREMENT	Egyedi azonosító	-
title	VARCHAR(255)	Film címe	-
description	TEXT	Részletes leírás	-
director	VARCHAR(100)	Rendező	NULL
duration	INT	Hossz percben	-
release_year	INT	Megjelenés éve	NULL
genre	VARCHAR(100)	Műfaj(ok)	NULL

Mező	Típus	Leírás	Alapértelmezett
rating	DECIMAL(3,1)	Értékelés (0-10)	NULL
poster_url	VARCHAR(500)	Poszter URL	NULL
trailer_url	VARCHAR(500)	Trailer URL	NULL
active	BOOLEAN	Aktív-e	TRUE
created_at	DATETIME	Létrehozás ideje	CURRENT_TIMESTAMP

4.2.3 screenings – Vetítések

Mező	Típus	Leírás	Alapértelmezett
id	INT AUTO_INCREMENT	Egyedi azonosító	-
movie_id	INT	Film azonosító	-
screening_date	DATE	Vetítés dátuma	-
screening_time	TIME	Vetítés időpontja	-
hall_number	INT	Terem száma	-
price	DECIMAL(10,2)	Jegyár	-
available_seats	INT	Szabad helyek száma	-

4.2.4 tickets – Jegyek

Mező	Típus	Leírás	Alapértelmezett
id	INT AUTO_INCREMENT	Egyedi azonosító	-
user_id	INT	Felhasználó azonosító	-

Mező	Típus	Leírás	Alapértelmezett
screening_id	INT	Vetítés azonosító	-
seat_number	VARCHAR(10)	Helyszám (pl.: A05)	-
price_paid	DECIMAL(10,2)	Fizetett ár	-
payment_id	INT	Fizetés azonosító	NULL
payment_status	VARCHAR(20)	Fizetés státusza	'pending'
status	VARCHAR(20)	Jegy státusza	'active'
purchase_date	DATETIME	Vásárlás időpontja	CURRENT_TIMESTAMP

Státuszok:

- active – Aktív, érvényes jegy
- used – Felhasznált (beszkennelt)
- cancelled – Lemondott

4.2.5 comments – Kommentek

Mező	Típus	Leírás	Alapértelmezett
id	INT AUTO_INCREMENT	Egyedi azonosító	-
user_id	INT	Felhasználó azonosító	-
movie_id	INT	Film azonosító	-
comment	TEXT	Komment szövege	-
rating	INT	Értékelés (1-5)	-

Mező	Típus	Leírás	Alapértelmezett
comment_date	DATETIME	Komment időpontja	CURRENT_TIMESTAMP

4.2.6 payments – Fizetések

Mező	Típus	Leírás	Alapértelmezett
id	INT AUTO_INCREMENT	Egyedi azonosító	-
user_id	INT	Felhasználó azonosító	-
screening_id	INT	Vetítés azonosító	-
stripe_session_id	VARCHAR(255)	Stripe session ID	-
stripe_payment_intent	VARCHAR(255)	Stripe payment intent	NULL
amount	INT	Összeg fillérben	-
seats	JSON	Kiválasztott helyek JSON	-
status	VARCHAR(20)	Fizetés státusza	'pending'
paid_at	DATETIME	Fizetés időpontja	NULL
created_at	DATETIME	Létrehozás ideje	CURRENT_TIMESTAMP

Fizetési státuszok:

- pending – Függőben
- paid – Sikeresen kifizetve

- failed – Sikertelen
 - refunded – Visszatérítve
-

5. Modulok és funkciók részletes leírása

5.1 Felhasználókezelő modul

Fájlok: login.php, register.php, logout.php, config.php

5.1.1 Regisztráció

A regisztrációs folyamat ellenőrzi:

- Kötelező mezők kitöltését
- Jelszó hosszát (minimum 6 karakter)
- Jelszavak egyezőségét
- Felhasználónév és email egyediségét

Kód részlet:

```
// register.php
if(strlen($password) < 6) {
    $errors[] = "A jelszónak legalább 6 karakter hosszúnak kell lennie!";
}

$hashed_password = password_hash($password, PASSWORD_DEFAULT);
$stmt = $pdo->prepare("INSERT INTO users (username, email, password, full_name) VALUES (?, ?, ?, ?)");
```

5.1.2 Bejelentkezés

A bejelentkezés során:

- Felhasználónév vagy email alapján keres
- password_verify() függvénnnyel ellenőrzi a jelszót
- Session-ben tárolja a felhasználó ID-t
- Frissíti az utolsó bejelentkezés időpontját

Kód részlet:

```
// login.php
if($user && password_verify($password, $user['password'])) {
    $_SESSION['user_id'] = $user['id'];

    $stmt = $pdo->prepare("UPDATE users SET last_login = NOW() WHERE id = ?");
    $stmt->execute([$user['id']]);
}
```

5.2 Filmek modul

Fájlok: movies.php, movie.php

5.2.1 Filmek listázása

A movies.php oldal:

- Oldaltördeléssel jeleníti meg a filmeket (12/24/48/96 film/oldal)
- Élő keresést biztosít cím, leírás, rendező alapján
- Műfaj szerinti szűrést tesz lehetővé
- AJAX alapú frissítést használ a szűréshez

AJAX végpont:

```
// movies.php (AJAX kérés kezelése)
if(isset($_GET['ajax']) && $_GET['ajax'] == 1) {
    header('Content-Type: application/json');

    // Keresési feltételek összeállítása
    $sql = "SELECT * FROM movies WHERE active = 1";
    $params = [];

    if(!empty($search)) {
        $sql .= " AND (title LIKE ? OR description LIKE ? OR director LIKE ?)";
        $params = array_merge($params, ["%$search%", "%$search%", "%$search%"]);
    }

    // Eredmények visszaküldése JSON-ben
    echo json_encode([
        'movies' => $movies,
        'total' => $total_movies,
        'page' => $page,
        'total_pages' => $total_pages
    ]);
}
```

5.2.2 Film részletei

A movie.php oldal:

- Megjeleníti a film összes adatát
- Listázza a következő 7 nap vetítéseit
- Megjeleníti a kommenteket és értékeléseket
- Lehetővé teszi új komment hozzáadását

Komment hozzáadás:

```
// movie.php
if($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['add_comment']) &&
$current_user) {
    $comment = trim($_POST['comment']);
    $rating = intval($_POST['rating']);

    if(!empty($comment) && $rating >= 1 && $rating <= 5) {
```

```

        $stmt = $pdo->prepare("INSERT INTO comments (user_id, movie_id, comment, rating) VALUES
        (?, ?, ?, ?)");
        $stmt->execute([$current_user['id'], $movie_id, $comment, $rating]);
    }
}

```

5.3 Vetítések modul

Fájlok: screenings.php, index.php

5.3.1 Vetítések listázása

A screenings.php oldal:

- Listázza az összes jövőbeli vetítést
- Szűrhető dátum és filmcím alapján
- Megjeleníti a vetítések részletes adatait

5.3.2 Heti műsor

Az index.php oldal:

- Megjeleníti a következő 7 nap vetítéseit
- Napokra bontva, tabokkal
- Automatikusan generálja a vetítéseket, ha hiányoznak

Automatikus generálás:

```

// config.php
function generateWeeklyScreenings($pdo) {
    for ($i = 0; $i < 7; $i++) {
        $date = date("Y-m-d", strtotime("+ $i days"));

        // Ha nincs 5 vetítés, generáljunk
        if ($count < 5) {
            // Véletlenszerű filmek kiválasztása
            shuffle($movies);
            $selected_movies = array_slice($movies, 0, 5);

            // Vetítések beszúrása
            for ($j = 0; $j < 5; $j++) {
                $insert_stmt->execute([
                    $selected_movies[$j],
                    $date,
                    $times[$j],
                    rand(1, 5),
                    2200,
                    120
                ]);
            }
        }
    }
}

```


5.4 Foglalási modul

Fájlok: booking.php, tickets.php

5.4.1 Ülésrend megjelenítése

A booking.php oldal:

- Dinamikusan generálja az ülésrendet (8 sor × 15 hely)
- Megkülönbözteti a szabad, foglalt és kiválasztott helyeket
- JavaScript segítségével kezeli a helyek kiválasztását

Ülésrend generálás:

```
// booking.php
$seat_rows = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'];
$seat_numbers_per_row = 15;

for($i = 0; $i < count($seat_rows); $i++) {
    $row = $seat_rows[$i];
    for($j = 1; $j <= $seat_numbers_per_row; $j++) {
        $seat = $row . str_pad($j, 2, '0', STR_PAD_LEFT);
        $is_occupied = in_array($seat, $occupied_seats);
        // HTML generálás
    }
}
```

5.4.2 Helyek kiválasztása

JavaScript részlet:

```
// booking.php
let selectedSeats = [];
const MAX_SEATS = 6;

function toggleSeat(seatNumber, element) {
    const index = selectedSeats.indexOf(seatNumber);

    if(index === -1) {
        if(selectedSeats.length >= MAX_SEATS) {
            alert(`Maximum ${MAX_SEATS} helyet választhat ki!`);
            return;
        }
        selectedSeats.push(seatNumber);
        element.classList.add('selected');
    } else {
        selectedSeats.splice(index, 1);
        element.classList.remove('selected');
    }

    updateSelectedSeatsDisplay();
}
```

5.5 Fizetési modul

Fájlok: stripe_payment.php, stripe_success.php, test_stripe.php

5.5.1 Stripe fizetés indítása

A stripe_payment.php:

- Ellenőrzi a kiválasztott helyek szabad voltát
- Kiszámolja a végösszeget (fillérben)
- Létrehoz egy Stripe Checkout Session-t
- Elmenti a fizetési rekordot az adatbázisba

Stripe session létrehozás:

```
// stripe_payment.php
$session = \Stripe\Checkout\Session::create([
    'payment_method_types' => ['card'],
    'line_items' => [[
        'price_data' => [
            'currency' => 'huf',
            'product_data' => [
                'name' => 'Mozi jegy(ek) - ' . $_POST['movie_title'],
                'description' => 'Helyek: ' . implode(', ', $selected_seats),
            ],
            'unit_amount' => $total_amount_filler, // FILLÉRBEN!
        ],
        'quantity' => 1,
    ]],
    'mode' => 'payment',
    'success_url' => 'http://' . $_SERVER['HTTP_HOST'] .
'/stripe_success.php?session_id={CHECKOUT_SESSION_ID}',
    'cancel_url' => 'http://' . $_SERVER['HTTP_HOST'] . '/booking.php?screening=' . $screening_id,
    'metadata' => [
        'screening_id' => (string)$screening_id,
        'seats' => implode(', ', $selected_seats),
        'user_id' => (string)$current_user['id']
    ]
]);
```

5.5.2 Sikeres fizetés kezelése

A stripe_success.php:

- Lekéri a Stripe session adatait
- Ellenőrzi, hogy a helyek még szabadok-e
- Létrehozza a jegyeket az adatbázisban
- Frissíti a szabad helyek számát

Tranzakció kezelés:

```
// stripe_success.php
$db->beginTransaction();

try {
    // Jegyek beszúrása
    foreach($selected_seats as $seat_number) {
```

```

    $stmt->execute([...]);
}

// Szabad helyek számának frissítése
$stmt = $pdo->prepare("UPDATE screenings SET available_seats = available_seats - ? WHERE id
= ?");
$stmt->execute([count($selected_seats), $screening_id]);

$stmt->commit();
} catch (Exception $e) {
    $pdo->rollBack();
    // Refund
}

```

5.6 Admin modul

Fájlok: admin.php, admin_action.php

5.6.1 Admin felület

Az admin.php oldal:

- Csak admin szerepkörű felhasználók számára elérhető
- Dashboard statisztikákkal (filmek, felhasználók, vetítések, jegyek, bevétel)
- Tabokra bontott menü (filmek, vetítések, felhasználók, jegyek, kommentek)
- CRUD műveletek minden entitáson

Jogosultság ellenőrzés:

```

// admin.php
if(!$current_user || $current_user['role'] != 'admin') {
    header("Location: index.php");
    exit();
}

```

5.6.2 Admin műveletek

Az admin_action.php kezeli:

- Film hozzáadás, módosítás, törlés
- Vetítés hozzáadás, módosítás, törlés
- Felhasználó módosítás
- Komment törlés

Film törlés (kapcsolódó rekordok ellenőrzésével):

```

// admin_action.php
if($_GET['action'] == 'delete_movie' && isset($_GET['id'])) {
    // Ellenőrizzük, van-e kapcsolódó vetítés
    $stmt = $pdo->prepare("SELECT COUNT(*) FROM screenings WHERE movie_id = ?");
    $stmt->execute([$_GET['id']]);
    $has_screenings = $stmt->fetchColumn();

    if($has_screenings > 0) {

```

```

$_SESSION['error'] = "Nem lehet törölni a filmet, mert vannak hozzá kapcsolódó vetítések!";
header("Location: admin.php?tab=movies");
exit();
}

// Töröljük a kommenteket
$stmt = $pdo->prepare("DELETE FROM comments WHERE movie_id = ?");
$stmt->execute([$id]);

// Töröljük a filmet
$stmt = $pdo->prepare("DELETE FROM movies WHERE id = ?");
$stmt->execute([$id]);
}

```

6. API és külső szolgáltatások

6.1 Stripe API integráció

A SzalkaCinema a Stripe fizetési szolgáltatást használja a bankkártyás fizetések kezelésére.

6.1.1 Stripe telepítés

```
composer require stripe/stripe-php
```

6.1.2 Stripe konfiguráció

```

// config.php
define('STRIPE_PUBLISHABLE_KEY', 'pk_test_...');
define('STRIPE_SECRET_KEY', 'sk_test_...');

require_once 'vendor/autoload.php';
\Stripe\Stripe::setApiKey(STRIPE_SECRET_KEY);

```

6.1.3 Fizetési session létrehozása

```

$session = \Stripe\Checkout\Session::create([
    'payment_method_types' => ['card'],
    'line_items' => [[
        'price_data' => [
            'currency' => 'huf',
            'product_data' => [
                'name' => 'Mozi jegy(ek)',
                'description' => 'Helyek: A01, A02',
            ],
        ],
        'unit_amount' => 4400, // 4400 fillér = 44 Ft
    ]],
    'quantity' => 1,
    'mode' => 'payment',
    'success_url' => 'https://.../stripe_success.php?session_id={CHECKOUT_SESSION_ID}',
]);

```

```
'cancel_url' => 'https://.../booking.php?screening=123',
]);
```

6.1.4 Teszkártya számok

Kártyatípus	Szám	Leírás
Sikeres fizetés	4242 4242 4242 4242	Visa, bármilyen jövőbeli dátum, bármilyen CVC
Sikertelen fizetés	4000 0000 0000 9995	Visa, kártya elutasítva
3D Secure	4000 0000 0000 3220	Visa, 3D Secure hitelesítés szükséges

6.2 AJAX API

A rendszer belső AJAX végpontokat használ a filmek szűréséhez és oldaltördeléséhez.

6.2.1 AJAX végpont

URL: movies.php?ajax=1

Paraméterek:

- search – Keresési kifejezés
- genre – Műfaj szűrő
- per_page – Oldalméret (12, 24, 48, 96)
- page – Oldalszám

Válasz formátum:

```
{
  "movies": [
    {
      "id": 1,
      "title": "A keresztapa",
      "description": "...",
      "director": "Francis Ford Coppola",
      "duration": 175,
      "release_year": 1972,
      "genre": "Dráma, Krimi",
      "rating": 9.2,
      "poster_url": "..."
    }
  ],
  "total": 25,
  "page": 1,
  "total_pages": 3,
  "per_page": 12
}
```

7. Biztonsági megfontolások

7.1 Adatbázis biztonság

7.1.1 SQL injection elleni védelem

A rendszer PDO prepared statement-eket használ minden adatbázis lekérdezéshez:

```
// HELYES - prepared statement
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");
$stmt->execute([$username]);

// HELYTELEN - SQL injection veszély
$stmt = $pdo->query("SELECT * FROM users WHERE username = '$username'");
```

7.1.2 Jelszavak tárolása

A jelszavak biztonságos, bcrypt alapú hasheléssel kerülnek tárolásra:

```
// Regisztráció
$hashed_password = password_hash($password, PASSWORD_DEFAULT);

// Bejelentkezés ellenőrzés
if(password_verify($password, $user['password'])) {
    // Sikeres bejelentkezés
}
```

7.2 XSS elleni védelem

Minden felhasználó által megadott adat HTML entity-ként kerül megjelenítésre:

```
// HELYES
<?php echo htmlspecialchars($user['full_name']); ?>

// HELYTELEN - XSS veszély
<?php echo $user['full_name']; ?>
```

7.3 Session biztonság

7.3.1 Session konfiguráció

```
// Ajánlott session beállítások
ini_set('session.cookie_httponly', 1);
ini_set('session.use_only_cookies', 1);
ini_set('session.cookie_secure', 1); // Ha HTTPS
```

7.3.2 Session védelem

```
// Session ID újragenerálás bejelentkezéskor
session_regenerate_id(true);
```

7.4 Jogosultság kezelés

Minden admin oldal és művelet ellenőrzi a felhasználó szerepkörét:

```
// Admin oldal
if(!$current_user || $current_user['role'] != 'admin') {
    header("Location: index.php");
    exit();
}

// Admin művelet
if($_GET['action'] == 'delete_movie') {
    // Jogosultság ellenőrzés itt is
}
```

7.5 Fájrendszer biztonság

7.5.1 .htaccess védelem

```
apache
Másolás
Letöltés
# Érzékeny fájlok védelme
<FilesMatch "\.(sql|log|ini|conf)$">
    Order allow,deny
    Deny from all
</FilesMatch>

# Könyvtárak listázásának tiltása
Options -Indexes
```

7.5.2 Konfigurációs fájlok

A config.php tartalmazza az adatbázis jelszavakat és Stripe kulcsokat. Ez a fájl soha nem lehet nyilvánosan elérhető.

8. Hibakeresés és naplózás

8.1 Hibanaplózás

8.1.1 PHP hibanaplózás

```
// config.php
ini_set('display_errors', 0); // Éles környezetben
ini_set('log_errors', 1);
ini_set('error_log', '/path/to/php-error.log');
```

8.1.2 Egyedi hibanaplózás

```
try {
    // Valamilyen művelet
} catch (Exception $e) {
    error_log("Hiba a fizetés során: " . $e->getMessage());
    $_SESSION['error'] = "Hiba történt!";
}
```

8.2 Stripe hibakezelés

```
try {
    $session = \Stripe\Checkout\Session::create([...]);
} catch (\Stripe\Exception\CardException $e) {
    error_log("Kártya hiba: " . $e->getError()->message);
    $_SESSION['error'] = "Kártya hiba: " . $e->getError()->message;
} catch (\Stripe\Exception\InvalidRequestException $e) {
    error_log("Stripe hiba: " . $e->getMessage());
    $_SESSION['error'] = "Fizetési rendszer hiba!";
}
```

8.3 Tesztelő eszközök

8.3.1 Stripe tesztelő

A test_stripe.php fájl segít az adatbázis struktúra ellenőrzésében és javításában:

```
// Ellenőrzi a tickets tábla oszlopait
$stmt = $pdo->query("DESCRIBE tickets");
$existing_columns = [];
while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    $existing_columns[] = $row['Field'];
}

// Hiányzó oszlopok hozzáadása
$columns_to_add = [
    'payment_id' => "ALTER TABLE tickets ADD COLUMN payment_id INT NULL",
    'payment_status' => "ALTER TABLE tickets ADD COLUMN payment_status VARCHAR(20)
DEFAULT 'pending'",
    'status' => "ALTER TABLE tickets ADD COLUMN status VARCHAR(20) DEFAULT 'active'"
];
```

9. Teljesítmény optimalizálás

9.1 Adatbázis optimalizálás

9.1.1 Indexek használata

```
-- Gyakran keresett mezők indexelése
CREATE INDEX idx_movies_title ON movies(title);
CREATE INDEX idx_movies_active ON movies(active);
```



```
CREATE INDEX idx_screenings_date ON screenings(screening_date);
CREATE INDEX idx_tickets_user ON tickets(user_id);
CREATE INDEX idx_tickets_status ON tickets(status);
```

9.1.2 Lekérdezések optimalizálása

```
// Csak szükséges mezők lekérése
$stmt = $pdo->query("SELECT id, title FROM movies WHERE active = 1");

// Helyett:
$stmt = $pdo->query("SELECT * FROM movies WHERE active = 1");
```

9.2 Képek optimalizálása

```
// Kép URL-ek kezelése, ha a kép nem elérhető
'"; ?>">
```

9.3 Oldaltördelés

```
$per_page = 12;
$page = isset($_GET['page']) ? (int)$_GET['page'] : 1;
$offset = ($page - 1) * $per_page;

$sql = "SELECT * FROM movies WHERE active = 1 LIMIT $per_page OFFSET $offset";
```

9.4 AJAX betöltés

A filmek listája AJAX segítségével frissül, így nem kell az egész oldalt újratölteni:

```
fetch(`movies.php?ajax=1&search=${searchValue}`)
    .then(response => response.json())
    .then(data => updateMovies(data));
```

10. Kódolási konvenciók

10.1 PHP kódolási szabályok

10.1.1 Változók elnevezése

```
// snake_case változónevek
$user_id = 123;
$full_name = "Kiss János";
$movie_title = "A keresztapa";
```

```
// Tömbök
$movies = [];
```

```
$selected_seats = [];
```

10.1.2 Függvények elnevezése

```
// camelCase függvénynevek  
function getCurrentUser() { ... }  
function generateDailyScreenings($pdo) { ... }  
function updateSelectedSeatsDisplay() { ... }
```

10.1.3 Osztályok (ha lesznek)

```
// PascalCase osztálynevek  
class Movie { ... }  
class User { ... }  
class Booking { ... }
```

10.2 HTML/CSS kódolási szabályok

10.2.1 CSS osztályok

```
/* BEM (Block Element Modifier) szerű elnevezés */  
.movie-card { }  
.movie-card__title { }  
.movie-card--featured { }  
  
.booking-container { }  
.booking-form { }  
.booking-form__submit { }
```

10.2.2 HTML struktúra

Futtatás

```
<div class="movie-card">  
  <div class="movie-card__poster-container">  
      
  </div>  
  <div class="movie-card__content">  
    <h3 class="movie-card__title">Film címe</h3>  
    <p class="movie-card__description">Leírás...</p>  
  </div>  
</div>
```

10.3 JavaScript kódolási szabályok

10.3.1 Változók elnevezése

```
// camelCase változónevek  
let selectedSeats = [];  
const MAX_SEATS = 6;  
let totalPrice = 0;  
  
// Függvények  
function toggleSeat(seatNumber) { ... }  
function updateSelectedSeatsDisplay() { ... }
```

10.3.2 Eseménykezelés

```
// Inline eseménykezelők helyett  
document.querySelectorAll('.seat').forEach(seat => {  
    seat.addEventListener('click', function() {  
        toggleSeat(this.dataset.seat, this);  
    });  
});
```

10.4 Adatbázis konvenciók

10.4.1 Táblák elnevezése

- Többes szám, kisbetűk: users, movies, screenings, tickets

10.4.2 Mezők elnevezése

- snake_case: user_id, full_name, screening_date, available_seats

10.4.3 Külső kulcsok

- user_id → users.id
- movie_id → movies.id
- screening_id → screenings.id

10.5 Fájl struktúra

- Minden PHP fájl <?php nyitótaggal kezdődik
- Nincs záró ?> tag a fájl végén (elkerüli a whitespace problémákat)
- Függvények és osztályok külön fájlokba szervezve (ha sok van)